Software Impacts 20 (2024) 100632

Contents lists available at ScienceDirect

Software Impacts

journal homepage: www.journals.elsevier.com/software-impacts

Original software publication

GraphIdx: An efficient indexing technique for accelerating graph data mining

Mostofa Kamal Rasel *, Mohammad Rezwanul Huq, Mohammad Arifuzzaman

Department of Computer Science and Engineering, East West University, Bangladesh

ARTICLE INFO

Keywords:
Graph index
Graph mining
Asynchronous I/Os

ABSTRACT

Many graph mining algorithms process large graphs with several passes and suffers from huge I/O cost. GraphIdx, an open-source C library, facilitates a memory-efficient indexing of large graphs to reduce that I/O cost. GraphIdx indexes a block of graph data for a set of nodes based on the empirical evaluation of edges. Due to the indexed graph, graph mining algorithms can access and process only the related nodes and their edges instead of scanning entire graph. As a result, the number of I/Os is significantly reduced. Moreover, GraphIdx accredited algorithms can process graphs in parallel due to the indexed data.

Code metadata

Current code version	v1.0
Permanent link to code/repository used for this code version	https://github.com/SoftwareImpacts/SIMPAC-2024-55
Permanent link to Reproducible Capsule	https://codeocean.com/capsule/6015981/tree/v1
Legal Code License	MIT License
Code versioning system used	git
Software code languages, tools, and services used	C; GCC compiler
Compilation requirements, operating environments & dependencies	stdlib.h, string.h
If available Link to developer documentation/manual	https://github.com/mmkrasel/graphidx/blob/main/README.md
Support email for questions	mostofa.kamal@ewubd.edu

1. Introduction

Graph mining algorithms, such as [1–5], usually perform several scans over the graph dataset to get the final results, which causes a large number of I/O requests. We know that the I/O cost of an algorithm can be reduced significantly if the algorithm accesses and processes indexed data selectively instead of scanning the entire dataset. Since indexed data can be accessed asynchronously from disk, algorithms can issue asynchronous I/O requests to different indexed data blocks and process those data in parallel to reduce processing costs.

However, the data of a graph is structured and the structural integrity of a graph must be addressed to ensure the benefit of indexing a graph. In this article, we demonstrate the implementation of a graph indexing technique called *GraphIdx*, which preserves the structural integrity in the produced index entries for a given graph. A graph mining algorithm can adapt GraphIdx to index a graph for accessing the selective parts of the graph asynchronously and processing them in parallel with multiple cores. Several researches have been benefited by adapting the GraphIdx, which are discussed in Section 4 of this paper.

The mechanism of indexing a directed graph by adapting GraphIdx was elaborately described in iTri [3]. However, in this paper, we illustrate the indexing process of both undirected and directed graphs. Beside, we describe some of the most commonly used operations, such as constructing index table, evaluating index entry, and retrieving the indexed part of the graph for a given index entry.

2. Indexing graph dataset

The operations in a graph mining algorithm are usually related to the neighbor nodes. For example, triangle [3–5] or clique [2] listing algorithms find common neighbors of two nodes of a graph. Some

The code (and data) in this article has been certified as Reproducible by Code Ocean: (https://codeocean.com/). More information on the Reproducibility Badge Initiative is available at https://www.elsevier.com/physical-sciences-and-engineering/computer-science/journals.

* Corresponding author.
E-mail addresses: mostofa.kamal@ewubd.edu (M.K. Rasel), mrhuq@ewubd.edu (M.R. Huq), mazaman@ewubd.edu (M. Arifuzzaman).

https://doi.org/10.1016/j.simpa.2024.100632
Received 27 February 2024; Accepted 16 March 2024

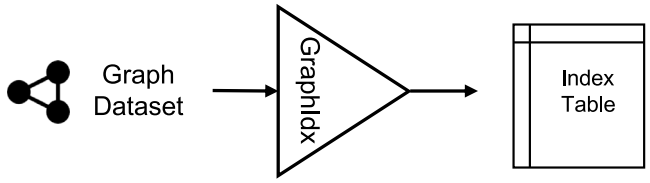

Fig. 1. A high-level overview of GraphIdx to index an input graph.

algorithms probe the existence of an edge from one node to another, which needs to get the source node information of an incoming edge in a directed graph. On the other hand, the PageRank [1] algorithm computes the weight of a node v based on the weights of the nodes X that have edges to v. These indicate that we need either to scan the directed graph to get X or store X of each node. We note that scanning a graph repeatedly or storing the incoming edges for every node is expensive in terms of both I/O and CPU cost. GraphIdx reduces the cost of storing the information about the incoming edges by constructing an index table for a group of nodes instead of every node. Fig. 1 illustrates the high-level overview of the GraphIdx.

Given an input graph, either undirected or directed, GraphIdx constructs an index table that holds the adjacent blocks list and an offset in each row. GraphIdx stores the index table on the disk after construction; however, the index table is loaded in the memory before starting a graph mining algorithm.

2.1. Indexing technique in brief

Let G denotes an input graph. Let V and E denote the sets of nodes and edges of G, respectively. GraphIdx assumes that $\forall v \in V$ are sorted in ascending order based on their Ids. On the other hand, this article considers that graph G is represented using the adjacency list. However, we note that GraphIdx can be adapted for indexing even if G is represented using the edge list.

GraphIdx assumes a graph G of several subgraphs such that each subgraph contains an equal number of nodes. Fig. 2 shows a directed graph with five subgraphs, each of them is represented by a data block. The figure also shows that every subgraph has five nodes except the last one. Let r denotes the number of nodes that exist in each subgraph. For a given graph G and r, GraphIdx continues to evaluate a block containing r nodes in each subgraph. Therefore, the last block represents a subgraph with $\leq r$ nodes. The aforementioned properties of the subgraphs indicate that the representative block id b for a node $v \in V$ can be evaluated as $b = \lfloor \frac{v}{r} \rfloor$ and the number of subgraphs in G as $n = \lceil \frac{|V|}{r} \rceil$.

If a subgraph in block b_o has a node that has an incoming edge from a node of a subgraph in block b_i, then b_i is called an *incoming block* of b_o and b_o is called an *outgoing block* of b_i. On the other hand, GraphIdx indexes a subgraph g for all nodes in g. An index entry is created in the index table for each subgraph. The row position of an entry in the index table represents the id of a block that contains a subgraph. Each entry of the index table represents a list of adjacent incoming blocks of a block and a pointer to that block in the graph dataset. GraphIdx can construct an index table by a single scan over the graph dataset, which is represented by either an adjacency list or an edge list. Fig. 2 shows the constructed index table for the given directed graph using the adjacency list. In this figure, the first column of the index table shows the list of adjacent incoming blocks for each subgraph. The second column represents the starting location of a block in the graph dataset. Furthermore, the row position on the left column outside the table represents the relative block ids.

Fig. 3 shows the indexing of an undirected graph using GraphIdx. Since the connected block id b for a node v can be obtained directly by $b = \lfloor \frac{v}{r} \rfloor$, GraphIdx does not store the connected block ids in the index table for indexing an undirected graph. However, each entry in the index table contains the starting location of each relative block of the graph dataset.

The index table contains n index entries, which can be evaluated from $n = \lceil \frac{|V|}{r} \rceil$. Thus, the relationship between n and r is inversely proportional. The size of the index table depends on both the value of r and the density of the graph. If the density of the graph increases, then the number of connected incoming blocks for a block also increases. As a result, the size of the index table also increases. However, we note that the overall size of an index table is negligible compared to the large graph as reported in the experiments of iTri [3].

2.2. Operations of GraphIdx

To construct the index table I, GraphIdx takes as input an input graph G, the direction type of G, and the value r that denotes the number of nodes in each subgraph. Based on the direction type G and r, GraphIdx constructs and stores the index table I.

GraphIdx reads I in memory before starting a graph mining algorithm. To access the source nodes of incoming edges to a node v, GraphIdx first evaluates the id b of the block that contains v using $b = \lfloor \frac{v}{r} \rfloor$. Then, for a directed graph, GraphIdx uses b to obtain the list of incoming blocks from I. Once the connected block ids are obtained, GraphIdx can access the nodes of each block using the starting location of the block. The size of a selected block having id b can be obtained from the difference of offsets of the current block and the next block having id $b+1$. We note that once the data of a selected incoming block B for a node v is accessed, we can probe an incoming edge from node u of B to v by checking the existence of v in the adjacency list of u.

3. Implementation

GraphIdx provides functionalities for constructing, storing, accessing, and probing into the index table for a given graph G. We have implemented GraphIdx using C language and share it as a C-header file (graphidx.h). The usages of the functions are illustrated in the following section.

3.1. Code example

```
#include<stdlib.h>
#include "graphidx.h"
void main(){
  ....
  /* Configure GraphIdx
    Graph dataset information
    - Represented using adjacency list
    - Data in binary format */
  char* dataset_path = "....";
  char* index_table_path = "....";
  int t = 1;    // Graph type: 0=undirected, 1=directed
  int r = ...;  // #nodes in a subgraph
  int N = ...;  // Max node id in the graph
  init_graphidx(t, r, N, dataset_path, index_table_path);
  ...
  /* Construct Index Table */
  construct_index_table();
  ...
  ...
  /* Load Index Table from disk to memory */
  load_index_table();
  ...
  /* Get incoming block ids for a given node */
  int v = ...;
  int num_blocks = 0;
  int* i_block_ids = get_incoming_blocks(v, &num_blocks);
   ...
  /* Get incoming node ids to a given node */
  int to_node = ...;
  int in_degree = 0;
  int* i_node_ids = get_incoming_nodes(to_node, &in_degree);
   ...
```

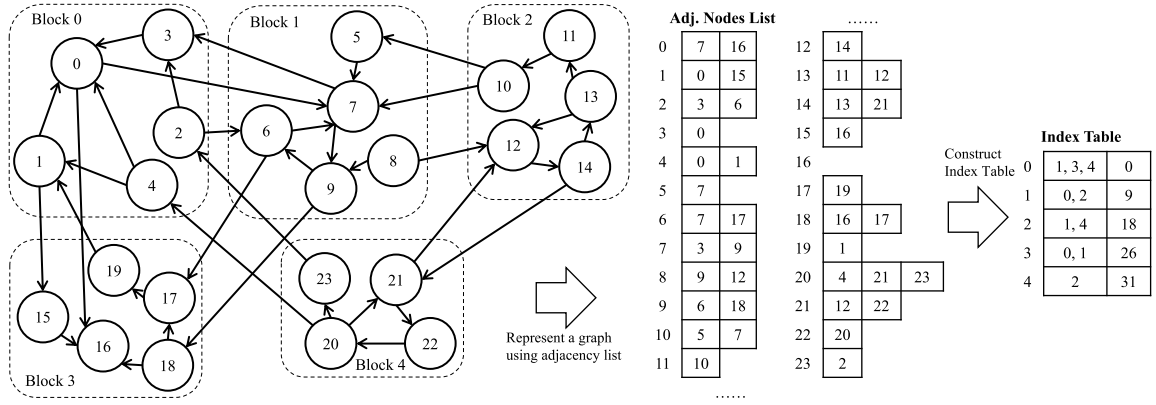

Fig. 2. Indexing a directed graph using GraphIdx.

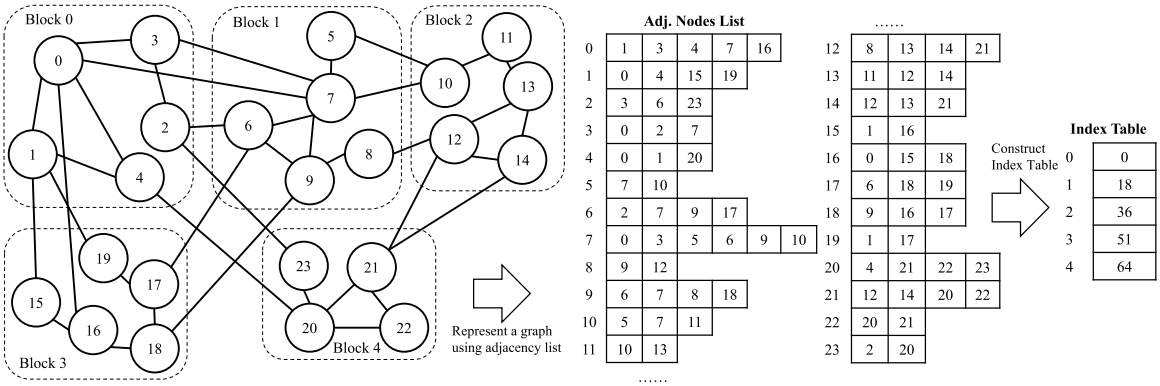

Fig. 3. Indexing an undirect graph using GraphIdx.

```
  /* Get outgoing node ids from a given node */
  int from_node = ...;
  int out_degree = 0;
  int* o_node_ids = get_outgoing_nodes(from_node, &out_degree);
  ...
  /* Probe that node 'u' is a source of
    an incoming edge to node v
    - 0: Negative, 1:Positive */
  int u = ...;
  int result = is_outneighbor(u, v);
  ...
}
```

4. Impact of GraphIdx

GraphIdx impacts the performance of graph mining algorithms primarily by allowing access to a part of graph data selectively, instead of performing a scan on the dataset. Since GraphIdx enables to process graph data without accessing the entire dataset, graph algorithms can reduce a sizeable number of I/Os. We note that less number of I/Os can significantly minimize the overall running time of an algorithm. In addition, since the parts of the graph dataset can be accessed asynchronously from an indexed graph, many sequential graph mining algorithms can be converted to parallel algorithms by accessing and processing the indexed graph data simultaneously.

The triangle listing algorithm, MGT [6] finds triangles by performing several scans on the graph datasets. However, the triangle list algorithms, such as iTri [3] and some improved versions of iTri [4,5] proposed triangle listing algorithms that used GraphIdx to index the directed graphs to reduce the I/O cost. iTri reported that the number of I/Os can be reduced over four times in some graph datasets compared to MGT. Moreover, iTri showed that if iTri processes indexed graphs sequentially, it becomes over two times faster than MGT due to the reduced I/O cost. On the other hand, if iTri performs in parallel by accessing indexed graph data asynchronously, the running times are reduced by about three times compared to a sequential version of iTri. On the other hand, GraphIdx-based iTri that accesses and processes indexed graph data in parallel can outperform MGT by about eight times for some datasets.

5. Limitations and future work

GraphIdx indexes a graph only if the graph is sorted based on its nodes and edges. The indexing of a graph can be embedded with the pre-processing of some graph mining algorithms, i.e. the algorithms discussed in [3–5] embed GraphIdx to index graph during the conversion of the graph from undirected to directed. However, an additional scan may be required for some other mining algorithms. GraphIdx indexes a data block of a fixed number of nodes that make the size of data block variables. In the future, we aim to update GraphIdx to index fixed size data blocks.

CRediT authorship contribution statement

Mostofa Kamal Rasel: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Resources, Software, Supervision, Validation, Writing – original draft. **Mohammad Rezwanul Huq:** Formal analysis, Validation, Writing – review & editing. **Mohammad Arifuzzaman:** Validation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] L. Page, S. Brin, R. Motwani, T. Winograd, The Pagerank Citation Ranking: Bring Order to the Web, Tech. rep., Technical report, stanford University, 1998.
[2] J. Cheng, Y. Ke, A.W.-C. Fu, J.X. Yu, L. Zhu, Finding maximal cliques in massive networks, ACM Trans. Database Syst. 36 (4) (2011) 21.
[3] M.K. Rasel, Y. Han, J. Kim, K. Park, N.A. Tu, Y.-K. Lee, itri: Index-based triangle listing in massive graphs, Inform. Sci. 336 (2016) 1–20.
[4] M.K. Rasel, Y.-K. Lee, Exploiting CPU parallelism for triangle listing using hybrid summarized bit batch vector, in: Big Data and Smart Computing (BigComp), 2016 International Conference on, IEEE, 2016, pp. 183–190.
[5] M.K. Rasel, E. Elena, Y.-K. Lee, Summarized bit batch-based triangle listing in massive graphs, Inform. Sci. 441 (2018) 1–17.
[6] X. Hu, Y. Tao, C.-W. Chung, Massive graph triangulation, in: Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data, 2013, pp. 325–336.